Where Software Eats the Car: Finding — and Proving — a Wedge in the Software-Defined Vehicle Stack

Sponsors: [Faculty Sponsor], Department of Electrical Engineering and Computer Science; [EcoCAR Faculty Advisor], UTK EcoCAR Team (liaison to the Product Innovation Track)

Computer Science, Computer Engineering • Product Innovation Track • Scored across Studio 1 (Dec 3, 2026) and Studio 2 (Apr 15, 2027)

Project Background and Objective

Vehicles are becoming software-defined: features once fixed in hardware now ship as updatable software, compute is consolidating into a handful of domain controllers, and value is migrating from the mechanical bill of materials toward software platforms, over-the-air updates, telematics, data, and cybersecurity. The competition's Product Innovation Track asks teams to do what a technical founder does: map that ecosystem, understand how incumbents capture value, find a defensible entry point — a "wedge" — where a small team's capability creates advantage, and test whether it survives contact with the engineers who would have to buy it. Read casually this looks like a business assignment. It is not. The decisive question is technical: where are the real engineering bottlenecks, and which are tractable? This project pairs ecosystem analysis with a prototype that tests the most load-bearing assumption under the leading hypothesis, using the EcoCAR vehicle and its data as the development substrate.

Project Scope

- **Ecosystem and value chain analysis.** Diagram the actor types — OEMs, Tier 1 suppliers, compute vendors, platform providers, fleet operators, consumers — and map the chain across compute, OTA, platforms, ADAS, telematics, cybersecurity, and data, identifying what protects performers and where competition is thin.
- **Engineering pain points.** Identify the technical bottlenecks at each stage. This is where technical background is decisive and where most teams have nobody qualified in the room.
- **Incumbent analysis and wedge formulation.** Researched Business Model Canvases for two companies at different stages, sources cited; screen attractive positions against what the team can build; state two to three wedge hypotheses as testable claims with critical assumptions explicit.
- **Technical validation prototype.** Identify the assumption that would kill the leading hypothesis, then build and evaluate a prototype testing it against real data or a realistic workload, with quantitative success criteria declared before building.
- **Customer discovery.** Structured interviews with practicing engineers to test whether the pain point is real and what the adoption barriers are; revise or abandon the hypothesis on the evidence.

Technical Considerations

- **A wedge without a prototype is an opinion.** The technical validation is the spine and the analysis exists to aim it. Scope the build so it can be completed and evaluated in one semester.
- **Negative results are results.** If the critical assumption fails, documenting why — with evidence — beats a demo resting on a hypothesis nobody tested. Students should be told this at the start.
- **Interviews are a measurement instrument and can be biased.** Ask about past behavior, not future intentions. Confidentiality boundaries are real: OEM data carries restrictions and the business portfolio does not permit it. Ideally the prototype is something the team keeps using.

Deliverables

- Ecosystem diagram and value chain map with engineering pain points identified at each stage.
- Two researched Business Model Canvases with citations, a feasibility screening analysis, and engineering persona definitions.
- Hypothesis and assumptions registry holding two to three wedge hypotheses with critical technical assumptions made explicit.
- Working prototype with code, documentation, and evaluation against pre-declared criteria; customer discovery findings; report, presentation, and a short public-facing insight artifact.

Assumptions and Dependencies

Requires access to the team's vehicle data — CAN logs, simulation outputs, requirements artifacts, toolchain — depending on the wedge selected, standard software development infrastructure, and subject matter experts for interviews, sourced through the competition's connections to Stellantis, MathWorks, and Argonne plus UTK's industry network. Analysis proceeds in the fall and the prototype is built in the spring, with wedge selection as the gate; the specific stack is settled by the end of the first term. Students need software engineering, data analysis, and technical writing proficiency.

Support and Guidance

Mentorship on software architecture, systems design, and evaluation methodology from the faculty sponsor, with ecosystem analysis, business model reasoning, and discovery technique covered by the competition's innovation-track training. The program provides direct access to practicing engineers at Stellantis, MathWorks, and Argonne — real domain experts to interview and a real vehicle program to build against.